Reporte Técnico de Actividades Práctico-Experimentales Nro. 5

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Anthony Yaguana
Asignatura	Simulación
Ciclo	5
Unidad	U2
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad..
Práctica Nro.	5
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	José O. Guamán Q.
Fecha	29 de Octubre de 2026 30 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

- Comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG.
- Implementar un generador LCG en Python.
- Implementar el mismo algoritmo en Java.
- Comparar las secuencias generadas y analizar sus propiedades.

3. Materiales, Reactivos, Equipos y Herramientas

- Computador.
- Python.
- Java/JDK
- Python
- JDK.
- Sistema operativo Linux o macOS

4. Procedimiento / Metodología Ejecutada

El Generador Congruencial Lineal utiliza la siguiente relación recursiva:

$$X(n+1) = (a \cdot X(n) + c) \bmod m$$

Los valores normalizados se obtienen mediante:

$$U(n) = X(n) / m$$

1. Seleccionar a = 1664525, c = 1013904223, m = 2^{32} y semilla = 12345.

```
================================================================================
  GENERADOR CONGRUENCIAL LINEAL (LCG) - Implementación en Python
  X(n+1) = (a * X(n) + c) mod m          U(n) = X(n) / m
================================================================================


--------------------------------------------------------------------------------
  Paso 1. Parámetros del generador
--------------------------------------------------------------------------------
  +-----------+-------------------+-------------------+
  | Parámetro |    Descripción    |       Valor       |
  +-----------+-------------------+-------------------+
  | a         | Multiplicador     |           1664525 |
  | c         | Incremento        |        1013904223 |
  | m         | Módulo            | 4294967296 (2^32) |
  | X(0)      | Semilla           |             12345 |
  | N         | Valores a generar |             10000 |
  +-----------+-------------------+-------------------+
```

2. Pasos 2 y 3: Generacion de 10000 valores en Python e implementar el mismo algoritmo en Java.

```
--------------------------------------------------------------------------------
  Pasos 2 y 3. Generación de 10000 valores en Python
--------------------------------------------------------------------------------
  Se generaron X(1) ... X(10000) con la semilla 12345.
  Último valor: X(10000) = 2793649609.
```

```
sh-5.3$ flatpak-spawn --host java -cp out lcg.Main
================================================================================
  GENERADOR CONGRUENCIAL LINEAL (LCG) - Implementación en Java
  X(n+1) = (a * X(n) + c) mod m          U(n) = X(n) / m
================================================================================

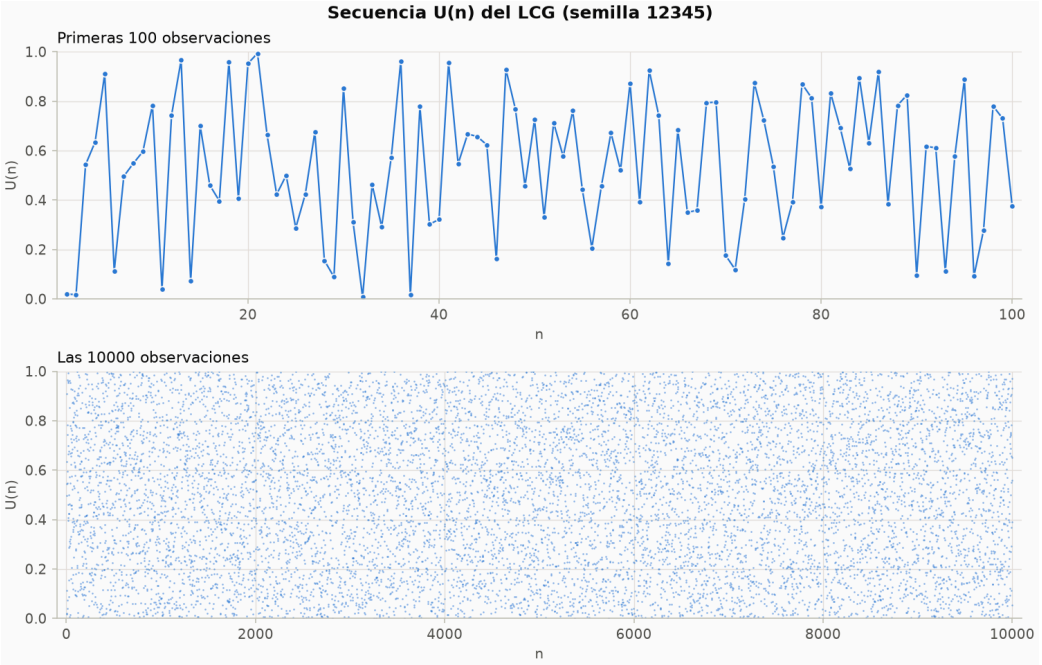
--------------------------------------------------------------------------------
  Paso 1. Parámetros del generador
--------------------------------------------------------------------------------
  +-----------+-------------------+-------------------+
  | Parámetro |    Descripción    |       Valor       |
  +-----------+-------------------+-------------------+
  | a         | Multiplicador     |           1664525 |
  | c         | Incremento        |        1013904223 |
  | m         | Módulo            | 4294967296 (2^32) |
  | X(0)      | Semilla           |             12345 |
  | N         | Valores a generar |             10000 |
  +-----------+-------------------+-------------------+

--------------------------------------------------------------------------------
```

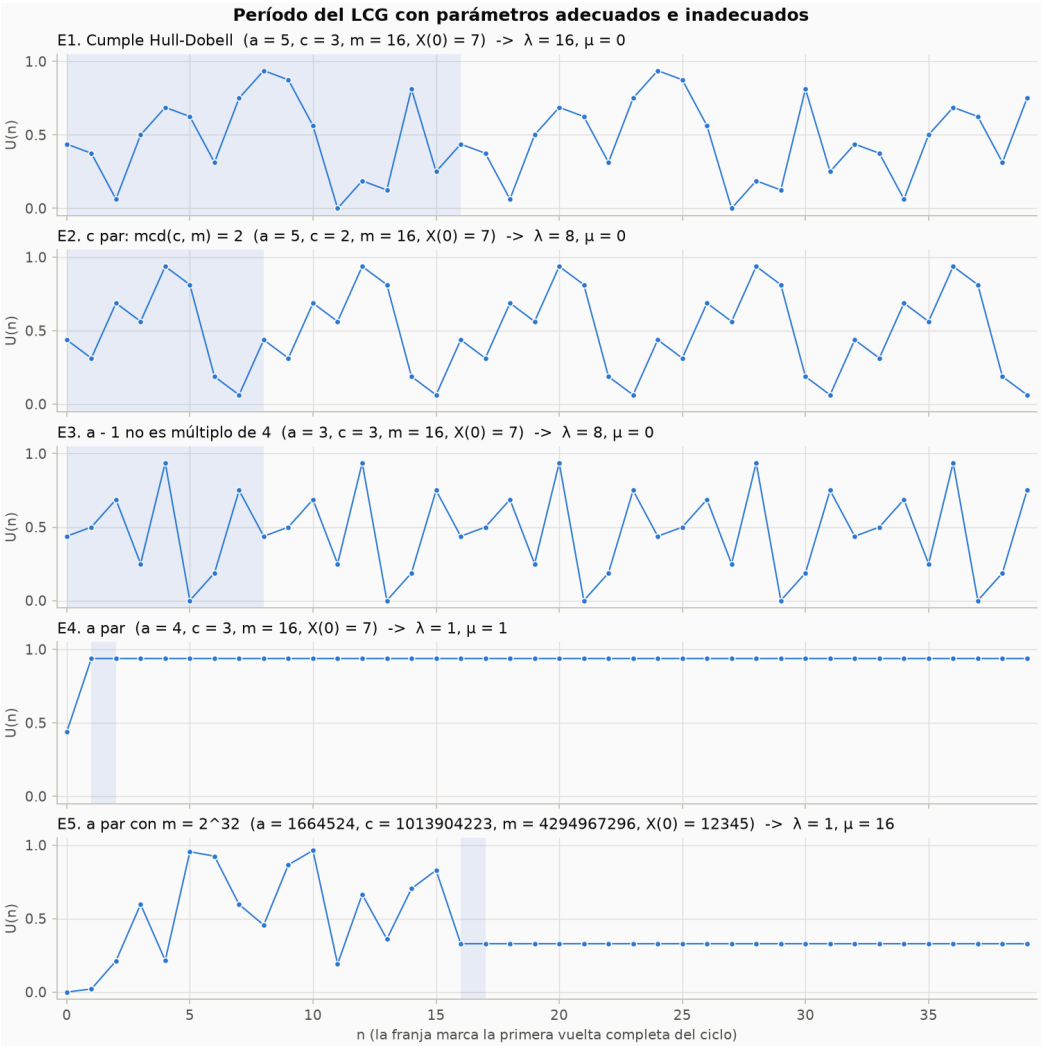
3. Comparación de los primeros 20 resultados: Python vs Java

```
--------------------------------------------------------------------------------
  Paso 4. Primeras 20 observaciones: Python vs Java
--------------------------------------------------------------------------------
  +----+-------------+------------+-------------+-------------+------------+
  | n  | X(n) Python | X(n) Java  | U(n) Python |  U(n) Java  | ¿Coincide? |
  +----+-------------+------------+-------------+-------------+------------+
  |  1 |    87628868 |   87628868 | 0.0204026857 | 0.0204026857 | Sí         |
  |  2 |    71072467 |   71072467 | 0.0165478482 | 0.0165478482 | Sí         |
  |  3 |  2332836374 | 2332836374 | 0.5431557945 | 0.5431557945 | Sí         |
  |  4 |  2726892157 | 2726892157 | 0.6349040561 | 0.6349040561 | Sí         |
  |  5 |  3908547000 | 3908547000 | 0.9100295138 | 0.9100295138 | Sí         |
  |  6 |   483019191 |  483019191 | 0.1124616691 | 0.1124616691 | Sí         |
  |  7 |  2129828778 | 2129828778 | 0.4958894053 | 0.4958894053 | Sí         |
  |  8 |  2355140353 | 2355140353 | 0.5483488443 | 0.5483488443 | Sí         |
  |  9 |  2560230508 | 2560230508 | 0.5961001171 | 0.5961001171 | Sí         |
  | 10 |  3364893915 | 3364893915 | 0.7834504161 | 0.7834504161 | Sí         |
  | 11 |   171172990 |  171172990 | 0.0398543174 | 0.0398543174 | Sí         |
  | 12 |  3194601925 | 3194601925 | 0.7438012224 | 0.7438012224 | Sí         |
  | 13 |  4148119648 | 4148119648 | 0.9658093676 | 0.9658093676 | Sí         |
  | 14 |   316399679 |  316399679 | 0.0736675409 | 0.0736675409 | Sí         |
  | 15 |  3004788882 | 3004788882 | 0.6996069294 | 0.6996069294 | Sí         |
  | 16 |  1976948425 | 1976948425 | 0.4602941743 | 0.4602941743 | Sí         |
  | 17 |  1702883732 | 1702883732 | 0.3964835154 | 0.3964835154 | Sí         |
  | 18 |  4121112547 | 4121112547 | 0.9595212869 | 0.9595212869 | Sí         |
  | 19 |  1744294886 | 1744294886 | 0.4061253010 | 0.4061253010 | Sí         |
  | 20 |  4092090893 | 4092090893 | 0.9527641565 | 0.9527641565 | Sí         |
  +----+-------------+------------+-------------+-------------+------------+
  Python vs Java: 10000 de 10000 posiciones coinciden (longitudes 10000 y 10000).
  Diferencia máxima en U(n): 0.0000000000
  Veredicto: las secuencias son IDÉNTICAS.
```

4. Graficar la secuencia de la semilla



5. Analizar el periodo

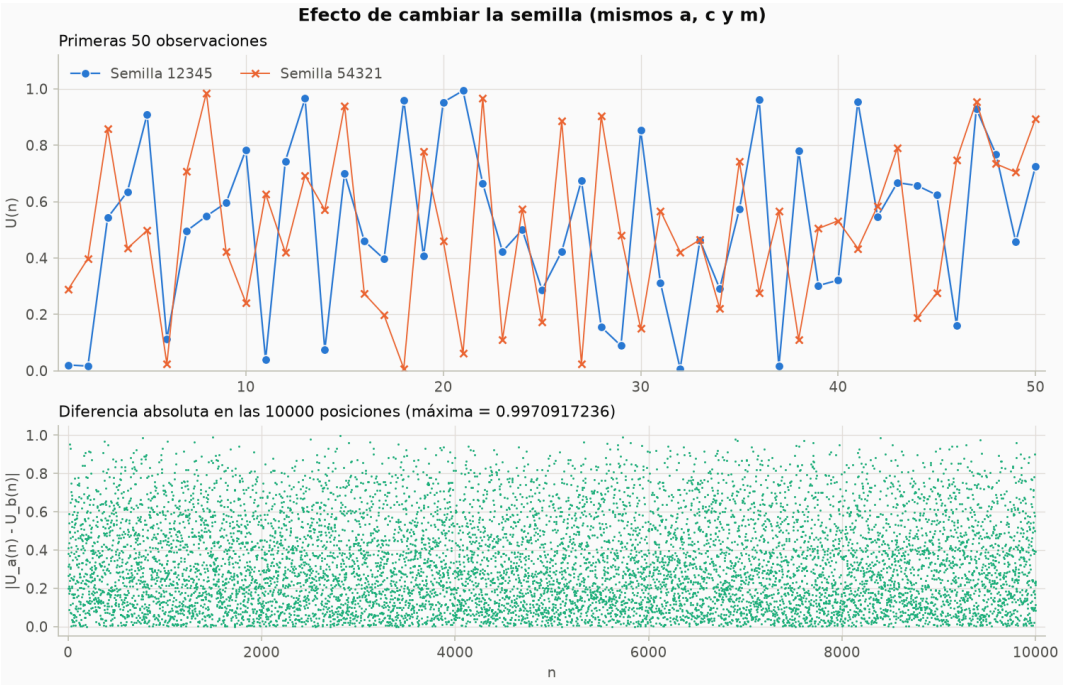


6. Cambio de semilla y comparación de las 2 semillas

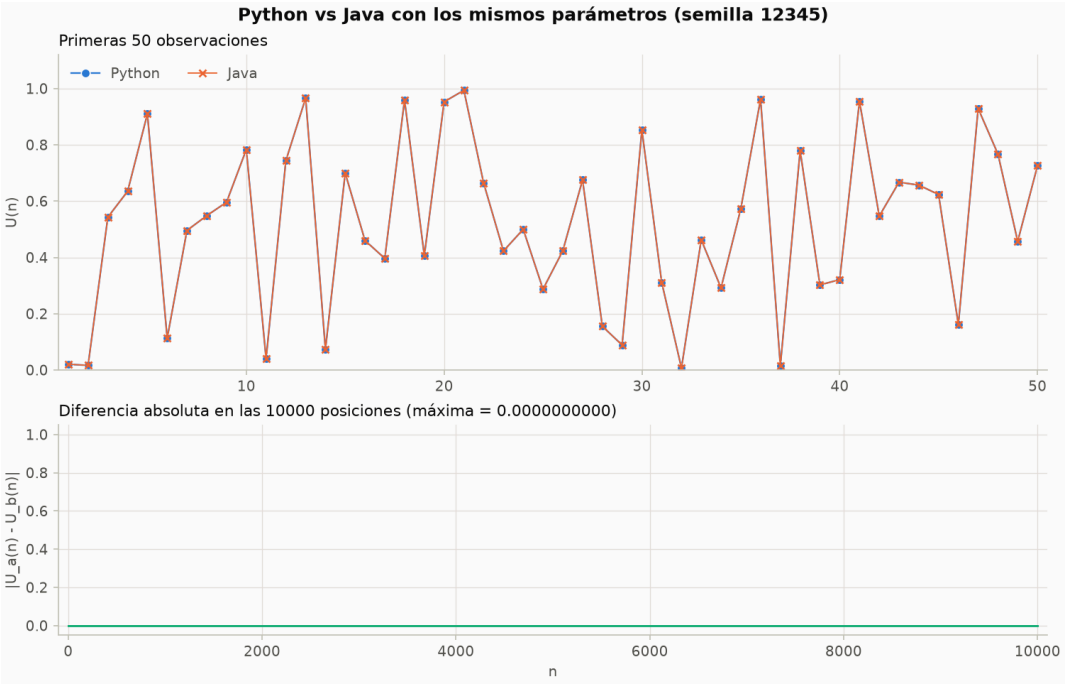
```
--------------------------------------------------------------------------------
  Paso 7. Cambio de semilla: 12345 -> 54321
--------------------------------------------------------------------------------
  Se conservan a, c y m; solo cambia el valor inicial X(0).
  +----+--------------+--------------+--------------+--------------+------------+
  | n  | X(n) s=12345 | X(n) s=54321 | U(n) s=12345 | U(n) s=54321 | ¿Coincide? |
  +----+--------------+--------------+--------------+--------------+------------+
  |  1 |     87628868 |   1238253532 | 0.0204026857 | 0.2883033669 | No         |
  |  2 |     71072467 |   1708513675 | 0.0165478482 | 0.3977943386 | No         |
  |  3 |   2332836374 |   3683344750 | 0.5431557945 | 0.8575955289 | No         |
  |  4 |   2726892157 |   1863498229 | 0.6349040561 | 0.4338794921 | No         |
  |  5 |   3908547000 |   2137457360 | 0.9100295138 | 0.4976655729 | No         |
  |  6 |    483019191 |    102299631 | 0.1124616691 | 0.0238184889 | No         |
  |  7 |   2129828778 |   3033777282 | 0.4958894053 | 0.7063563173 | No         |
  |  8 |   2355140353 |   4230855161 | 0.5483488443 | 0.9850727303 | No         |
  |  9 |   2560230508 |   1814797060 | 0.5961001171 | 0.4225403676 | No         |
  | 10 |   3364893915 |   1036872339 | 0.7834504161 | 0.2414156541 | No         |
  | 11 |    171172990 |   2695818966 | 0.0398543174 | 0.6276692650 | No         |
  | 12 |   3194601925 |   1801476157 | 0.7438012224 | 0.4194388532 | No         |
  | 13 |   4148119648 |   2976955512 | 0.9658093676 | 0.6931264680 | No         |
  | 14 |    316399679 |   2448971127 | 0.0736675409 | 0.5701955238 | No         |
  | 15 |   3004788882 |   4038571114 | 0.6996069294 | 0.9403031119 | No         |
  | 16 |   1976948425 |   1174362305 | 0.4602941743 | 0.2734275314 | No         |
  | 17 |   1702883732 |    849107756 | 0.3964835154 | 0.1976983054 | No         |
  | 18 |   4121112547 |     33496219 | 0.9595212869 | 0.0077989462 | No         |
  | 19 |   1744294886 |   3337365822 | 0.4061253010 | 0.7770410324 | No         |
  | 20 |   4092090893 |   1978253189 | 0.9527641565 | 0.4605979633 | No         |
  +----+--------------+--------------+--------------+--------------+------------+
  s=12345 vs s=54321: 0 de 10000 posiciones coinciden (longitudes 10000 y 10000).
  Diferencia máxima en U(n): 0.9970917236
  Veredicto: las secuencias son DISTINTAS (primera diferencia en n = 1).
```

5. Resultados

- **Entre semillas:** los valores son distintos, pero ambas muestras tienen media, varianza y desviación muy cercanas a las de U(0, 1), el período no cambia porque las condiciones de Hull-Dobell no dependen de la semilla.



- **Entre lenguajes:** con la semilla 54321, Python y Java también coinciden en 10000 de 10000 posiciones. Los estadísticos que imprime cada programa son idénticos hasta el último decimal.



Lenguaje	Facilidad de implementación del LCG	Velocidad	Resultados
Python	Fácil	Normal	Los mismos que en Java
Java	Más difícil que Python	Más veloz que Python	Los mismos que en Python

6. Preguntas de Control

- **¿Qué significa pseudoaleatorio?**
  Es un número o serie de números que aparentemente son aleatorios o al azar pero en realidad son generados con un algoritmo computacional determinista y son repetibles.

- **¿Qué función cumple la semilla?**
  Es el valor inicial X(0) y fija el punto del ciclo donde empieza la secuencia. La misma semilla reproduce exactamente la misma secuencia, lo que permite repetir los experimentos, y otra semilla produce una secuencia distinta con los mismos parámetros.

- **¿Qué significa período?**
  Es la cantidad de valores que el generador produce antes de que la secuencia empiece a repetirse. En un LCG el máximo es m; con estos parámetros se alcanza el período completo de $2^{32} = 4\,294\,967\,296$ valores.

- **¿Qué ocurre si se utilizan parámetros inadecuados?**
  El período se acorta, la secuencia se repite pronto o queda constante y aparecen patrones que rompen la uniformidad. En la práctica, con m = 16 el período bajó de 16 a 8 (c par, o a − 1 no múltiplo de 4) y a 1 (a par). Con $m = 2^{32}$, a = 1664524 volvió constante la secuencia a partir de X(16).

- **¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?**
  Permite verificar cada implementación con la otra: si ambas producen la misma secuencia, se descartan errores de programación y diferencias propias de cada lenguaje, como el desbordamiento de enteros, la división o el redondeo. También demuestra que el resultado depende solo del algoritmo y sus parámetros, y permite comparar el rendimiento: Java recorrió el período completo en 68 s, mientras que Python tardaría unos 33 minutos.

7. Conclusiones

- **Las dos implementaciones son equivalentes:** con los mismos parámetros, Python y Java produjeron la misma secuencia en las 10000 posiciones y con ambas semillas, y sus archivos de salida son idénticos. Los resultados dependen solo del algoritmo, de los parámetros y de la semilla inicial.
- **La igualdad del LCG exige cuidar detalles en cada lenguaje:** El algoritmo es el mismo, pero hubo que controlar el tamaño de los enteros, la división y el redondeo decimal.
- **La semilla cambia la secuencia de valores generados no su calidad:** Con la semilla 54321 no coincidió ningún valor, pero los estadísticos y el período fueron equivalentes. Con período completo, todas las semillas recorren el mismo ciclo de 2^{32} valores; la semilla solo elige el punto de partida para la generación.
- **Pseudoaleatorio no significa que sea azar verdadero:** La secuencia generada cumple con las pruebas de uniformidad e independencia, pero es predecible y tiene regularidades como la alternancia par e impar de X(n), por eso solo sirve para simulación y no sirve para criptografía.

8. Bibliografía / Referencias

- Diapositivas de la semana 5
- APE Guía de la semana 5

9. Anexos

- Repositorio: https://github.com/Anthony-YP/Simulacion.git

10. Declaración de uso de la IA

En el desarrollo de esta práctica se utilizó Claude (Anthropic), como asistente en las siguientes tareas:

1. Implementación del Generador Congruencial Lineal con Python y Java con arquitectura MVC.
2. Ejecución de la verificación: pruebas unitarias, comparación de las secuencias generadas por ambos lenguajes y medición del período del generador.
3. Resolución de problemas del entorno, como la ejecución de Java en el equipo de trabajo.

Mi participación consistió en definir los requisitos y criterios del proyecto, ejecutar los programas en mi equipo, validar los resultados obtenidos, sintetizar los resultados y llegar a la conclusión de la práctica.

Revisé completamente el contenido generado con apoyo de la IA y asumo la responsabilidad del trabajo entregado.